hblibArticle User Manual
for hblibArticle.sty and hblibCommon.sty v20261109T0615

Haluk Bingol

Contents

1. Getting started 2
2. Quick reference 2
2.1. Option 2
2.2. Code 3
2.3. Section numbering 3
2.4. References 3
2.5. Text and time stamps 3
2.6. Algorithms 3
2.7. Colors 4
3. Code 4
3.1. Typed code: javacode, bashcode 4
3.2. Code from a file: \javafile, \bashfile 4
3.3. Part of a file and its line numbers 5
3.4. Language tags 5
3.5. Other listing styles 5
3.6. Inline code 6
4. Lists, tables, trees, figures 6
4.1. Lists (enumitem) 6
4.2. Menus and keys (menukeys) 6
4.3. Folder trees (forest) 6
4.4. Algorithms (algorithm2e) 7
5. Data structures and code from files 7
5.1. Data structures (hblibDatastructures) 7
5.2. Code and output from files (hblibCodeoutput) 8
6. Section numbering 8
W. WarmUp 9
W1. A warm-up task 9
G. Goal 9
G1. First goal 9
G1.1. A detail 9
G2. Second goal 9
C. Challenge 9
C1. A challenge 9
H. Home 9
H1. At home 9
7. Macros 9
7.1. References 9
7.2. Text 9
7.3. Time stamps 9
8. Layout 9
8.1. Titles and headings 9
8.2. Table of contents 10
8.3. Footer 10
8.4. Pages and columns 10
8.5. Hyperlinks 10
9. Customizing 10
10. Troubleshooting 11

1. Getting started

```latex
\documentclass[11pt]{amsart}           % or article; [twocolumn] works too
\usepackage{hblibArticle}              % titles and headings as typed
\usepackage[amsheadings]{hblibArticle} % amsart: keep CAPITAL titles, small-caps headings
\usepackage{../../cis000_latex/lib/hblibArticle}   % from another folder
\usepackage{../../cis000_latex/lib/hblibDatastructures} % optional: arrays, stacks, trees
\usepackage{../../cis000_latex/lib/hblibCodeoutput}     % optional: code and output from files
```

Library	Contents
hblibCommon	shared by articles and slides: fonts, packages, algorithms, colors and emphasis, code styles and environments, `\hCode`, time stamps, `\reffig` ..., `\hAbs`, column type `L`; not loaded by the document, but by the two below
hblibArticle	articles: page, lists, titles as typed, contents, section numbering, footer, hyperlinks
hblibPresentation	slides (beamer): theme, `\hSection`, `\hSubsection`, `\hSeparator`, `\hSectionSkip`
hblibDatastructures	optional: arrays, stacks, queues, strings, grids, trees, commit graphs
hblibCodeoutput	optional: code and output read from files, in boxes (tcolorbox)

- Keep the five files in one folder: each library loads the others it needs from its own folder.
- Old documents with `\usepackage{hblibCommon}` need no change: hblibCommon then loads hblibArticle (the log says so).
- Load it with `\usepackage`, without `.sty`. `\input{hblibCommon.sty}` does *not* work: names with `@` inside the library break, and LaTeX reports "Missing \begin{document}".
- With a relative path LaTeX warns "You have requested package '../../cis000_latex/hblibCommon', but the package provides 'hblibCommon"'. The warning is harmless.
- Compile *twice*: the contents, the references and the "page / pages" in the footer are known only after the first run.
- Already loaded, so do not load again: fontenc (T1), lmodern, geometry, amsmath, amssymb, graphicx, xcolor, tikz, calc, array, booktabs, enumitem, forest, datetime2, menukeys, algorithm2e, listings, fancyhdr, lastpage, tocloft (not with amsart), hyperref.
- Page: A4 with 2 cm margins. Change it after `\usepackage{hblibArticle}`, e.g. `\geometry{bottom=2.5cm}`.

2. Quick reference

Every option, command and environment of hblibArticle and hblibCommon, with the section that shows an example. Optional arguments are in `[brackets]`.

2.1. Option

Write	Result
`\usepackage{hblibArticle}`	titles, authors and headings as typed (also with amsart)
`\usepackage[amsheadings]{hblibArticle}`	amsart: keep its CAPITAL title and small-caps headings

2.2. Code

Write	Result
`\begin{javacode}[options]`	typed Java: framed box, tag “java”
`\begin{bashcode}[options]`	typed bash: framed box, tag “bash”
`\javafile[options]{file}`	Java from a file, line numbers, tag “java”
`\bashfile[options]{file}`	bash from a file, tag “bash”
`\hCode{text}`	inline code: typewriter, colored, 2 pt smaller; escape `_ { } \`
`\hCodeJ{code}`	inline Java, syntax colored, typed as is
`\hCodeB{code}`	inline bash, typed as is
`\code{code}`	old name of `\hCodeJ`
`\hCodeTag{text}`	the next listing shows *text* in its upper right corner
`\hCodeTagsfalse`, `\hCodeTagstrue`	no tags from here on; tags again
`style=`*name*	listing styles: Table 1

Options are ordinary listings options, e.g. `numbers=left`, `firstline=5`, `lastline=7`, `firstnumber=5`, `basicstyle=\ttfamily \scriptsize`. Examples: Sec. 3.

2.3. Section numbering

Write	Result
`\hNumbersLetter{G}`	next section G; subsections G1, G2; subsubsections G1.1
`\hNumbersArabic`	back to 1, 1.1; the numbers continue where they stopped
`\hNumbersArabic[1]`	back to numbers; the next section is 1
`\hSectionWarmUp`	section “W WarmUp”
`\hSectionGoal`	section “G Goal”
`\hSectionChallenge`	section “C Challenge”
`\hSectionHome`	section “H Home”

Example: Sec. 6.

2.4. References

Write	Result	Write	Result
`\reffig{label}`	Fig. 3	`\refthm{label}`	Theorem 1
`\refeq{label}`	Eq. 2	`\refthmA{thm}{item}`	Theorem 1(b)
`\reftbl{label}`	Table 1	`\reflem{label}`	Lemma 2
`\refsec{label}`	Sec. 4	`\refdef{label}`	Definition 3
`\refalg{label}`	Alg. 1	`\refexmp{label}`	Example 4
`\reflst{label}`	List 2	`\refcite{key}`	Ref. [7]

2.5. Text and time stamps

Write	Result
`\hQuote{text}`	a short quotation: small, sans serif, in quotes
`\hIdea{text}`	a note to self: tiny, olive, in brackets
`\hFootnote{who}{text}`	a footnote starting with a red “@who :”
`\hAbs{x}`	$\lvert x \rvert$ with bars that fit
`\hFontLarger{N}{text}`	text N pt larger ($N < 0$: smaller); old name `\hbFontShift`
`\YULearn`	YULearn, the course web site
`\hbTimeStamp`	v2026-11-14, used in the footer
`\hbTimeStampDraft`	red “–Draft–” with date and time

2.6. Algorithms

algorithm2e is loaded with `ruled, vlined, linesnumbered` and captions in `\small`. hblibCommon adds `\Fn{name}{body}` (“function … end”) and the aligned `\Input{text}` and `\Output{text}` lines. Example: Alg. 1.

2.7. Colors

All can be changed with `\definecolor`, see Sec. 9.

Name	RGB	Used for
colorHCode	240, 80, 50	`\hCode`
codebg	248, 248, 248	background of code boxes
codekw	0, 70, 160	keywords
codestr	30, 130, 60	strings
codecmt	120, 120, 120	comments, line numbers, tags
codeframe	40, 90, 160	code box frame (as in the slide decks)
termbg, termfg, termframe	35, 38, 45 …	terminal: background, text, frame
termcmd	255, 214, 102	terminal: lines starting with $

3. Code

3.1. Typed code: javacode, bashcode

```latex
\begin{javacode}
public class Hello {   // Turkish letters work: ğüşıöç
    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
\end{javacode}
```

```java
public class Hello {   // Turkish letters work: ğüşıöç
    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

```latex
\begin{bashcode}
javac Hello.java   # compile
java Hello         # run
\end{bashcode}
```

```bash
javac Hello.java   # compile
java Hello         # run
```

Any listings option can be given in brackets, e.g. line numbers: `\begin{javacode}[numbers=left]`.

```java
1 int a = 1;
2 int b = a + 1;
```

3.2. Code from a file: \javafile, \bashfile

The whole file, with line numbers (`\javafile` numbers the lines by default):

```latex
\javafile{D.java}
```

```java
1 package demo;
2
3 /** D shows how line numbers work with firstline. */
4 public class D {
5     public static void main(String[] args) {
6         int n = args.length;          // line 6
7         System.out.println("n = " + n);
8     }
9 }
```

```latex
\bashfile{run.sh}
```

```bash
#!/bin/bash
# compile and run D
javac -d out D.java
```

```
java -cp out demo.D aaa bbb
```

3.3. Part of a file and its line numbers

Note. `\javafile[firstline=5, lastline=7]{D.java}` numbers the lines 1, 2, 3, not 5, 6, 7. Add `firstnumber=5` to the options when you want the file's own line numbers.

```latex
\javafile[firstline=5, lastline=7]{D.java}                % numbered 1, 2, 3
\javafile[firstline=5, lastline=7, firstnumber=5]{D.java}  % numbered 5, 6, 7
```

```java
    public static void main(String[] args) {
        int n = args.length;          // line 6
        System.out.println("n = " + n);
```

```java
    public static void main(String[] args) {
        int n = args.length;          // line 6
        System.out.println("n = " + n);
```

Now the text can say "line 6" and mean the same line as the file.

3.4. Language tags

As in markdown, a code box shows its language small and gray in its upper right corner.

- `bashcode` and `\bashfile` show **bash**.
- `javacode` and `\javafile` show **java**, also when you print part of a file with `firstline=...`.
- Other listings: put `\hCodeTag{...}` right before the listing. For example, `\hCodeTag{txt}` before a `MANIFEST.MF` box matches a markdown "txt" block.
- Turning them off: `\hCodeTagsfalse` in a document turns tags off from that point on; `\hCodeTagstrue` turns them on again.

```latex
\hCodeTag{txt}
\begin{lstlisting}[style=outputstyle, frame=single, rulecolor=\color{gray!40}]
Main-Class: MyClass
\end{lstlisting}
```

```txt
Main-Class: MyClass
```

```latex
\hCodeTagsfalse
\begin{bashcode}
ls -l    # no tag
\end{bashcode}
\hCodeTagstrue
```

```
ls -l    # no tag
```

Note. The tag takes no space of its own: it is drawn at the right end of the first line. A first line that runs the full width of the box goes under the tag.

3.5. Other listing styles

All of them are used as `\begin{lstlisting}[style=...]`.

Table 1. Listing styles of hblibCommon.

Style	Use
`codesnippet`	short code typed in the .tex: framed, light background (javacode and bashcode use it)
`codestyle`	colored code with line numbers, no frame (the default of a plain lstlisting)
`outputstyle`	program output: no colors, no numbers
`terminalstyle`	a terminal session: lines starting with $ are highlighted
`texsnippet`	LaTeX source
`javastyle, bashstyle`	codesnippet plus the language
`mystyle`	old name (hblibSimple, articles only); the same as codestyle

Table 1 lists them. A terminal session:

```latex
\hCodeTag{terminal}
\begin{lstlisting}[style=terminalstyle, frame=single, rulecolor=\color{termframe}, backgroundcolor=\color{termbg
    }, basicstyle=\ttfamily\footnotesize\color{termfg}]
$ java -jar MyApp.jar aaa
~~MyClass
i=0:aaa
\end{lstlisting}
```

```terminal
$ java -jar MyApp.jar aaa
~~MyClass
i=0:aaa
```

3.6. Inline code

```latex
\hCode{MyClass.java}            % plain, colored, 2 pt smaller: escape _ { } \ as in text
\hCode{project\_dir}            % gives project_dir
\hCodeJ{int n = s.length();}    % Java, syntax colored, typed as is
\hCodeB{echo 'aa bbb' >file}    % bash, typed as is (quotes stay straight)
```

Results: `MyClass.java`, `project_dir`, `int n = s.length();`, `echo 'aa bbb' >file`.

Note. `\hCodeJ` and `\hCodeB` cannot be used in section titles, captions or footnotes, and their text must not contain `%` or `#`. Use `\hCode` there, with the special characters escaped.

4. Lists, tables, trees, figures

4.1. Lists (enumitem)

```latex
\begin{enumerate}[label=\roman*., itemsep=0pt]   % i., ii.; no extra space between items
    \item first
    \item second
\end{enumerate}
```

i. first
ii. second

Other options: `[nosep]` (no space at all), `[label=(\alph*)]` for (a), (b), `[topsep=1pt]`.

4.2. Menus and keys (menukeys)

```latex
\menu{File > Save As}    \keys{Ctrl + C}    \keys{\shift + \cmd + B}
```

File › Save As Ctrl + C ⇧ + ⌘ + B

4.3. Folder trees (forest)

```latex
\begin{forest}
for tree={font=\ttfamily\small, grow'=0, folder, draw=none,
          s sep=0pt, inner ysep=1pt}            % less space between lines
    [project/
        [src/
            [packageA/ [modifyMe.java] [doNotModifyMe.java]]
        ]
    ]
\end{forest}
```

```
project/
└ src/
  └ packageA/
    ├ modifyMe.java
    └ doNotModifyMe.java
```

Note. `s sep` is the space between lines, `inner ysep` the padding around each name. Do not change `l sep`: with `folder` it breaks the connecting lines. `folder indent=.5em` makes the indent smaller.

4.4. Algorithms (algorithm2e)

```latex
\begin{algorithm}[H]
    \caption{isBalanced($s$)}\label{alg:balanced}
    \KwIn{a string $s$}
    \KwOut{true or false}
    $S \gets$ an empty stack\;
    \ForEach{character $c$ of $s$}{
        \lIf{$c$ is an opening bracket}{push $c$ on $S$}
        \ElseIf{$c$ is a closing bracket}{
            \lIf{$S$ is empty \textbf{or} pop($S$) does not match $c$}{\Return{false}}
        }
    }
    \Return{$S$ is empty}\;
\end{algorithm}
```

Algorithm 1: isBalanced(s)

Input: a string s
Output: true or false

```
1 S ← an empty stack;
2 foreach character c of s do
3     if c is an opening bracket then push c on S;
4     else if c is a closing bracket then
5         if S is empty or pop(S) does not match c then return false;
6 return S is empty;
```

Alg. 1 is placed here because of `[H]`. Without it, it floats like a figure.

Note. `\clearpage` prints all waiting figures, tables and algorithms, then starts a new page. To print them without a new page, use `\FloatBarrier` of the `placeins` package.

5. Data structures and code from files

Two optional libraries work in articles too; load them after hblibArticle:

```latex
\usepackage{../../cis000_latex/lib/hblibArticle}
\usepackage{../../cis000_latex/lib/hblibDatastructures}   % \hStack, \hQueue, \hArray, \hGrid, trees, commits
\usepackage{../../cis000_latex/lib/hblibCodeoutput}       % \lstcode, \lstcoderight, \lstterminal, ...
```

Their full manuals are hblibDatastructures-UserManual and hblibCodeoutput-UserManual (slide decks).

5.1. Data structures (hblibDatastructures)

Every macro is used inside a `tikzpicture`; put it in a figure to give it a caption and a label.

```latex
\begin{figure}[h]
  \centering
  \begin{tikzpicture}
    \hStack{1}{6}{4}{15,6,2,9}[0.9][0.6]
    \hArray[id=a, name=a, at={(8,0)}, cur={2}, done={0,1}]{3,1,4,1,5}
  \end{tikzpicture}
  \caption{A stack and an array.}\label{fig:ds}
\end{figure}
```

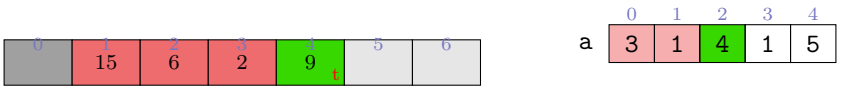
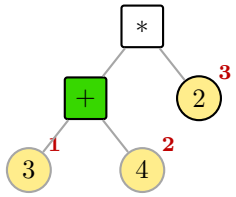


FIGURE 1. A stack and an array.

Fig. 1: the stack has top 4 (green); in the array, cell 2 is the current one and cells 0, 1 are done.

```latex
\begin{tikzpicture}[htree]
  \node[hnode, hsquare] {$*$}
    child {node[hnode, hsquare, hhot] {$+$}
      child {node[hnode, hleaf, horder=1] {3}}
      child {node[hnode, hleaf, horder=2] {4}}}
```

```
    child {node[hnode, hleaf, horder=3] {2}};
\end{tikzpicture}
```



5.2. Code and output from files (hblibCodeoutput)

The code and its output stay in files (paths relative to the .tex); the output is what the program really printed. In an article the *step macros show the output at once, and side by side means two top-aligned columns.

```
\lstcoderight[0.66][firstline=21, firstnumber=21, basicstyle=\ttfamily\scriptsize]
             {ArrayStack.java}{Java}{ArrayStack.txt}
```
latex

ArrayStack.java — Java

```
21          ArrayStack s = new ArrayStack();
22          for (int x : new int[] {15, 6, 2, 9}) s.push(x);
23          System.out.println("pop: " + s.pop());
24          System.out.println("pop: " + s.pop());
25          s.push(17);
26          while (!s.isEmpty()) System.out.print(s.pop() + " ");
27          System.out.println();
28      }
29  }
```

Output — ArrayStack.txt

```
pop: 9
pop: 2
17 6 15
```

Write	Shows
\lstcode[opts]{code}{Language}	code in a box with its file name
\lstoutput[opts]{output}	output in a dark box
\lstterminal[opts]{transcript}	a terminal session, $ lines highlighted
\lstcodedown, \lstcodedownstep	code, output below
\lstcoderight, \lstcoderightstep[width]	code left, output right

Note. A missing file gives a red box, not an error, so a document compiles before its outputs exist.

6. Section numbering

Sections are numbered 1, 2, … with subsections 1.1 and subsubsections 1.1.1. The style can switch before any \section:

```
\hNumbersLetter{G}        % next section: G, subsections G1, G2, subsubsections G1.1
\section{Group work}
\hNumbersArabic           % back to numbers; they continue where they stopped
\hNumbersArabic[1]        % back to numbers, restarting at 1
```
latex

Lab handouts use four ready-made sections:

```
\hSectionWarmUp      % W  WarmUp
\hSectionGoal        % G  Goal
\hSectionChallenge   % C  Challenge
\hSectionHome        % H  Home
```
latex

The next four sections are made with them, and the section after them with \hNumbersArabic. Look at their numbers here and in the contents.

Note. \ref prints the number in the current style, e.g. Sec. G1 and Sec. 6. Links in the PDF stay correct, even though G and a later section 7 have the same counter value.

W. WarmUp

W1. A warm-up task

Complete `lowerFirstChar`.

G. Goal

G1. First goal

G1.1. A detail

G2. Second goal

C. Challenge

C1. A challenge

H. Home

H1. At home

7. Macros

7.1. References

```latex
\reffig{fig:x}  \refeq{eq:x}  \reftbl{tbl:x}  \refsec{sec:x}  \refalg{alg:x}  \reflst{lst:x}
\refthm{thm:x}  \refthmA{thm:x}{item:b}  \reflem{lem:x}  \refdef{def:x}  \refexmp{ex:x}
\refcite{key}
```

They print “Fig. 3”, “Eq. 2”, “Table 1”, “Sec. 4”, “Alg. 1”, “List 2”, “Theorem 1”, “Theorem 1(b)”, “Lemma 2”, “Definition 3”, “Example 4”, “Ref. [7]”, with a non-breaking space. In this document: Table 1, Alg. 1, Sec. 3.4.

7.2. Text

```latex
\hQuote{Talk is cheap. Show me the code.}
\hIdea{add an example here}
text\hFootnote{HB}{check this}
\hAbs{x - y}
\YULearn
\hFontLarger{2}{two points larger}  \hFontLarger{-2}{two points smaller}
```

“Talk is cheap. Show me the code.” [add an example here] text[1] $|x - y|$ YULearn two points larger, two points smaller

Note. A document may define its own `\YULearn` with `\newcommand`; the library then uses the document’s.

7.3. Time stamps

```latex
\hbTimeStamp       % in the footer
\hbTimeStampDraft  % red, with the time: for drafts
```

2026-11-14 –Draft– v2026-11-14/10:59:35

8. Layout

8.1. Titles and headings

With `amsart`, the title, the authors and the section headings are printed as typed, not in capitals or small caps. `\usepackage[amsheadings]{hblibArticle}` keeps the `amsart` look.

[1] @HB : check this

8.2. Table of contents

One size smaller than the text; sections in bold; subsections indented, subsubsections more.

```latex
\setcounter{tocdepth}{2}   % after \usepackage{hblibArticle}: no subsubsections in the contents
```

8.3. Footer

Every page, also the first one with amsart: a rule, then on the left the file name and the date, on the right "page / pages". The file name is \jobname, the name of the .tex file.

8.4. Pages and columns

```latex
\newpage     % a new page (in two columns: a new column)
\clearpage   % a new page, after printing all waiting figures, tables and algorithms
\documentclass[11pt, twocolumn]{article}   % two columns
```

8.5. Hyperlinks

The contents, \ref and URLs are clickable. A red box marks an internal link, a blue box a URL; the boxes are shown on screen, not printed.

https://github.com/halukbingol/LaTeX-Templates

```latex
\hypersetup{hidelinks}                      % after \usepackage{hblibArticle}: no marks
\hypersetup{colorlinks=true, linkcolor=blue!70!black, urlcolor=blue, citecolor=green!50!black}
                                            % colored text instead of boxes
```

Note. \hCode in a section title prints correctly, but hyperref warns "Token not allowed in a PDF string" for the PDF bookmark. The warning is harmless.

9. Customizing

Put these after \usepackage{hblibArticle}; inside a group {...} they change only that part.

```latex
\geometry{margin=2.5cm}                  % margins
\definecolor{colorHCode}{RGB}{0,90,200}  % \hCode in blue
\definecolor{codekw}{RGB}{160,0,80}      % keywords in purple
\setcounter{tocdepth}{2}                 % no subsubsections in the contents
\hCodeTagsfalse                          % no language tags
\renewcommand{\hbTimeStamp}{v1.2}        % the version in the footer
```

One listing only: give the option in brackets.

```latex
\begin{javacode}[basicstyle=\ttfamily\scriptsize, numbers=left]
int x = 1;   // smaller, numbered
\end{javacode}
```

```java
1 int x = 1;   // smaller, numbered
```

A color only in a group: blue here, the usual color here.

10. Troubleshooting

Message or problem	Cause	Fix
Missing \begin{document} right after loading	`\input{hblibArticle.sty}`	`\usepackage{hblibArticle}`
File ‘hblibArticle.tex’ not found	`\input{hblibArticle}`	`\usepackage{hblibArticle}`
File ‘. . . /hblibCommon.sty’ not found	hblibCommon not next to hblibArticle	keep all hb libraries in one folder
Command \hEmph (or another name) already defined, right after loading	old and new hb libraries mixed	update all hb libraries together
You have requested package ‘../../. . . /hblibArticle’, but the package provides ‘hblibArticle’	a relative path in `\usepackage`	harmless
Footer shows “1 / ??”; contents or references empty or “??”	first run	compile again
Command \hNumbersArabic already defined	the document still has its own copy of the numbering commands	delete the copy
Command \code (or another name) already defined	the document defines a name the library has	rename it, or use `\renewcommand`
Token not allowed in a PDF string	`\hCode` in a section title	harmless
Package Listings Error: style codesnippet undefined	an old library (hblibSimple)	use hblibArticle
`\hCodeJ` fails in a title, caption or footnote	listings cannot work inside another command’s argument	`\hCode` there
`\javafile` with `firstline=5` is numbered 1, 2, 3	listings counts the printed lines	add `firstnumber=5`
the tag covers the end of the first code line	a full-width first line	shorten it, or `\hCodeTagsfalse` before the listing
TeX capacity exceeded, sorry [input stack size=10000] at `\maketitle` (amsart)	a size command such as `\normalsize` in `\title`; amsart copies the title to the page header	give a plain short title: `\title[Short]{Long \\ {\small ...}}`
a figure or algorithm appears pages later	floats wait for room	`[H]`, `\clearpage`, or `\FloatBarrier` (placeins)